

Svarark

Del A

#	Kode	Output	Hvorfor
1	<pre>t = ["a","b","c","d","e"] print(t[1]) print(t[-1]) print(t[1:3])</pre>	<pre>b e ['b', 'c']</pre>	Nul-indeksering og det eksklusive slutindeks i én opgave. <code>t[1]</code> er nummer to, ikke nummer et. I R ville <code>t[1]</code> give "a" og <code>t[-1]</code> give alt undtagen det første.
2	<pre>t = [1, 2, 3] print(t * 2) print(t + [4])</pre>	<pre>[1, 2, 3, 1, 2, 3] [1, 2, 3, 4]</pre>	Samme operatører som på tekst i lektion 2, samme opførsel: <code>*</code> gentager, <code>+</code> sætter sammen. Bemærk at <code>t + 4</code> ville fejle — der skal en liste til.
3	<pre>t = [3, 1, 2] print(t.sort()) print(t)</pre>	<pre>None [1, 2, 3]</pre>	Listemetoder ændrer listen på stedet og returnerer <code>None</code> . Det er præcis modsat strengmetoder. Konsekvensen: <code>t = t.sort()</code> sætter <code>t</code> til <code>None</code> og ødelægger alt derefter.
4	<pre>a = [1, 2, 3] b = a b[0] = 17 print(a)</pre>	<pre>[17, 2, 3]</pre>	<code>b</code> er ikke en kopi, men et nyt navn til den samme liste. Der findes kun ÉN liste i hukommelsen. En rigtig kopi laves med <code>b = a[:]</code> . I R ville <code>a</code> være uændret.
5	<pre>t = ("a") print(type(t)) u = ("a",) print(type(u))</pre>	<pre><class 'str'> <class 'tuple'></pre>	Kommaet laver tuplen — ikke parenteserne. Uden komma er <code>('a')</code> bare en tekst med parenteser omkring.
6	<pre>x, y = "en-to".split("-") print(y)</pre>	<pre>to</pre>	Tuple assignment: <code>.split()</code> giver to dele, og begge navngives i samme linje. Mønsteret bruges hele tiden, når felter skal skilles ad.
7	<pre>d = {"ida": 34} print("ida" in d) print(34 in d)</pre>	<pre>True False</pre>	<code>in</code> kigger kun på NØGLER. Vil man søge i værdierne, skal man skrive <code>34 in d.values()</code> .
8	<pre>d = {"a": 1} print(d.get("b", 0)) print(d["b"])</pre>	<pre>0 KeyError: 'b'</pre>	<code>.get()</code> giver en standardværdi, når nøglen mangler. Firkantparentesen fejler. Begge dele er nyttige: vil man tælle, vil man have 0; skal nøglen findes, vil man have fejlen.
9	<pre>total = 0 for t in [3, -1, 12, 999]: if t < 0 or t == 999: continue total = total + t print(total)</pre>	<pre>15</pre>	<code>3 + 12</code> . De to kodede værdier springes over. Det er øvelse 3.3 fra lektion 2 skrevet med <code>continue</code> i stedet for en ny liste.
10	<pre>alder = 12 if alder < 30: print("ung") elif alder < 18: print("barn")</pre>	<pre>ung</pre>	Betingelserne tjekkes oppefra og ned, og kun den første sande gren køres. "barn" er sandt, men bliver aldrig testet. Rettes ved at bytte om på de to grene.

Del B

Fem personer og fire kommuner. Aalborg findes kun i den ene tabel, Esbjerg kun i den anden.

#	Kald	Rækker	Hvem falder ud / kommer til
1	<code>merge(..., on="kommune")</code>	4	Standard er inner join. Aalborg findes ikke i kommuner og Esbjerg ikke i personer — begge ryger ud. Person 5 er væk uden en advarsel.
2	<code>how="left"</code>	5	Alle fem personer beholdes. Aalborg får NaN i region og indbyggere. Det er som regel det, man vil have i registerdata: man mister ikke observationer, man får synlige huller.
3	<code>how="right"</code>	5	Alle fire kommuner beholdes. København har to personer, så $2 + 1 + 1 + 1 = 5$. Esbjerg kommer med som en række uden nogen person.
4	<code>how="outer"</code>	6	De fire fra inner, plus Aalborg, plus Esbjerg. Nyttig til at inspicere, hvad der ikke matcher — sjældent til selve analysen.
5	<code>merge(personer, job, on="id")</code>	5	Person 1 har to brancher og person 3 har to. De øvrige har én hver. $2 + 1 + 2 = 5$ rækker fra tre personer; person 4 og 5 findes ikke i job og ryger ud. Person 1 og 3 tælles nu dobbelt i ethvert gennemsnit.
6	<code>pd.concat([runde1, runde2])</code>	0, 1, 0, 1	Rækkenumrene følger med fra hver tabel, så to rækker hedder 0 og to hedder 1. <code>ignore_index=True</code> giver dem 0-3 i stedet. Antal rækker er 4.
7	<code>ledighed.pivot(...)</code>	3 rækker 2 kolonner	Seks rækker i langt format (tre kommuner × to år) bliver til tre rækker med én kolonne per år. Ingen information tabt — den er flyttet fra rækker til kolonner.

Notebook: Fra tekst til tabel

1.1 Fra liste af dictionaries til tabel

```
print(tekster[0])
print(tekster[0].keys())

{'id': 1, 'parti': 'Parti A', 'aar': 2022, 'tekst': 'Vi vil ...'}
dict_keys(['id', 'parti', 'aar', 'tekst'])
```

Trin 3: svaret er (6, 4)

Seks rækker, fordi der er seks taler i listen. Fire kolonner, fordi hver dictionary har fire nøgler.

1.2 Ord i den første tekst

```
print(rens(tekster[0]["tekst"]))
print(len(rens(tekster[0]["tekst"])))
```

Output: 13 ord

1.3 Optælling på tværs af alle seks

```
antal = {}

for r in tekster:
    for o in rens(r["tekst"]):
        antal[o] = antal.get(o, 0) + 1

print(len(antal))
print(sum(antal.values()))
37
69
```

37 unikke ord ud af 69 i alt. Den ydre løkke går gennem teksterne, den indre gennem ordene i hver tekst. Vi kalder dette et nested loop.

1.4 De hyppigste ord

```
par = []
for nogle, vaerdi in antal.items():
    par.append((vaerdi, nogle))

par.sort(reverse=True)
print(par[:6])

[(7, 'i'), (6, 'vil'), (6, 'vi'), (6, 'investere'),
 (4, 'og'), (3, 'uddannelse')]
```

2.1 Antal ord per tale

Trin 3: de to pladser skal udfyldes sådan:

```
raekker = []

for r in tekster:
    raekker.append({"id": r["id"],
                    "antal_ord": len(reus(r["tekst"]))})

print(raekker[0])
print(len(raekker))

{'id': 1, 'antal_ord': 13}
6
```

Trin 4: den samme linje som i opgave 1.1:

```
ordtal = pd.DataFrame(raekker)
print(ordtal)

   id  antal_ord
0   1         13
1   2         11
2   3         11
3   4         13
4   5          9
5   6         12
```

2.2 Koblingen med partier

Trin 1: de to lister af partinavne:

```
['Parti A', 'Parti B', 'Parti C']
['Parti A', 'Parti B', 'Parti D']
```

Parti C har holdt taler, men står ikke i partitabellen. Parti D står i partitabellen, men har ikke holdt nogen tale.

Trin 2: inner join:

```
kobl = pd.merge(df, partier, on="parti")
print(len(df), "->", len(kobl))

6 -> 4
```

Parti C's to taler er væk. Ingen advarsel.

Trin 3 og 4:

```
inner 4
left   6
right  5
outer  7
```

2.3 Fælden

```
print(len(pd.merge(df, talere, on="parti")))
```

Output: 8

6 taler × 4 talere gav 8 rækker. Parti A har to taler, så hver af A's to taler dubleres. A fylder nu 4 af de 8 rækker, selvom partiet kun holdt en tredjedel af talerne.

```
pd.merge(df, talere, on="parti", validate="many_to_one")
```

MergeError: Merge keys are not unique in right dataset

pandas siger fra, når man har sagt hvad man forventede.

3.1 Fra langt til bredt

```
tael = pd.merge(df, ordtal, on="id")
bred = tael.pivot(index="parti", columns="aar",
                  values="antal_ord")
print(bred)
```

aar	2022	2023
parti		
Parti A	13	13
Parti B	11	9
Parti C	11	12

Tre rækker, to kolonner. Seks observationer er blevet til tre rækker uden tab af information.

3.2 Ændringen

```
bred["aendring"] = bred[2023] - bred[2022]
print(bred)
```

aar	2022	2023	aendring
parti			
Parti A	13	13	0
Parti B	11	9	-2
Parti C	11	12	1

Kolonnenavnene er tal, ikke tekst. `bred[2023]` virker, `bred["2023"]` giver `KeyError`.

3.3 Tilbage til langt format

```
lang = bred.reset_index().melt(
    id_vars="parti",
    var_name="aar",
    value_name="antal_ord")
print(len(lang))
```

Output: 6 rækker

`reset_index()` er nødvendig, fordi pivot lagde partinavnet i indekset, ikke i en kolonne. `melt` kan kun folde kolonner sammen, så indekset skal først gøres til en almindelig kolonne.